

Docker Containerization: Complete Learning Guide



Table of Contents

1. [Introduction to Docker](#)
2. [Docker vs Virtual Machines](#)
3. [Docker Architecture](#)
4. [Docker Images](#)
5. [Docker Containers](#)
6. [Building Docker Images](#)
7. [Running Docker Containers](#)
8. [Docker Commands Reference](#)
9. [Docker Volumes and Data Management](#)
10. [Docker Networks](#)
11. [Docker Compose](#)
12. [Best Practices](#)
13. [Hands-on Examples](#)
14. [Troubleshooting](#)

Introduction to Docker

What is Docker?

Docker is a revolutionary containerization platform that simplifies the process of creating, deploying, and running applications by using containers. These containers allow developers to package an application with all the parts it needs, such as libraries and other dependencies, and ship it all out as one package.

Why Docker?

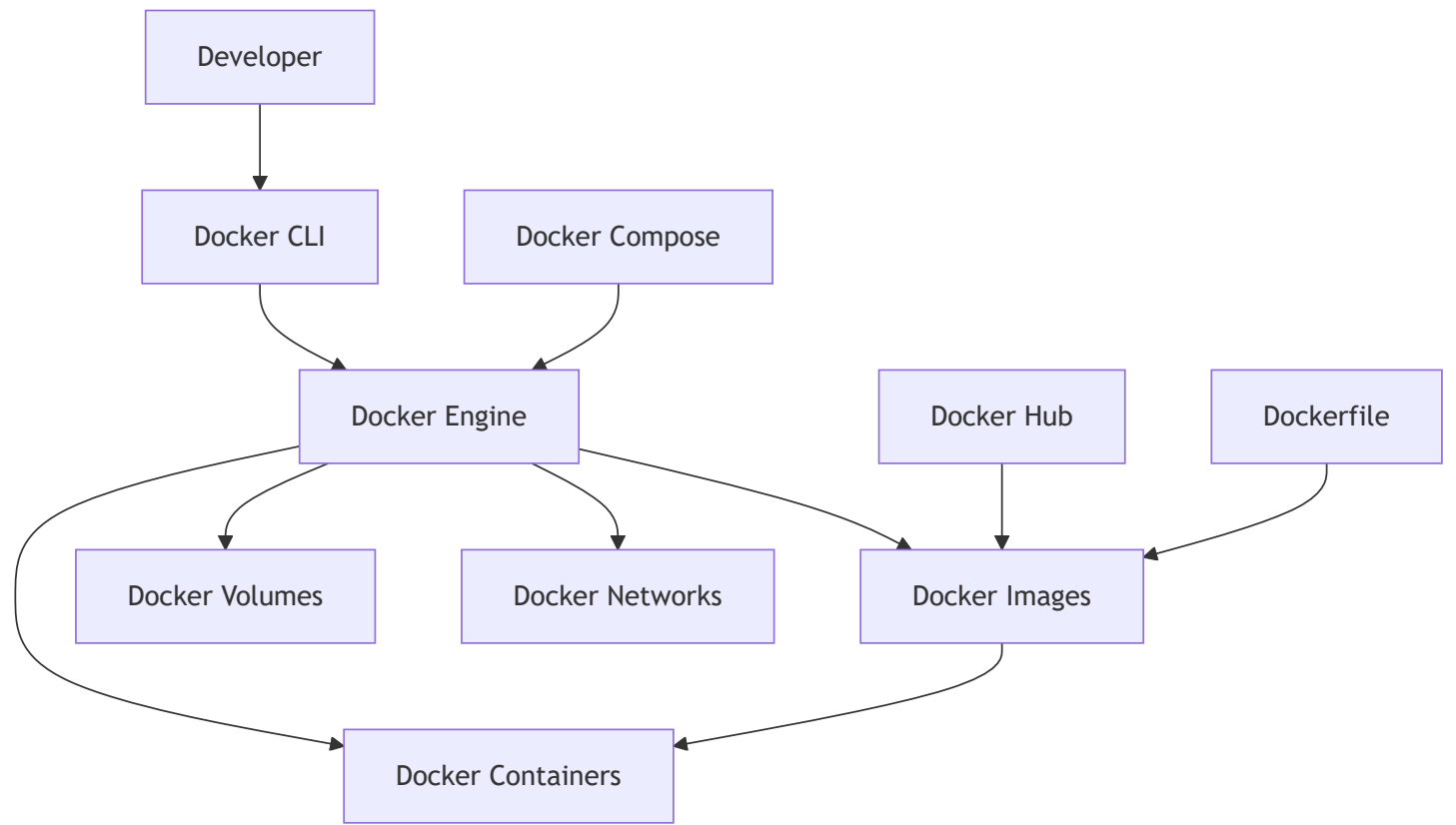
The "It Works on My Machine" Problem

Before Docker, developers often faced the frustrating scenario where applications worked perfectly in development but failed in production due to environmental differences.

Key Benefits

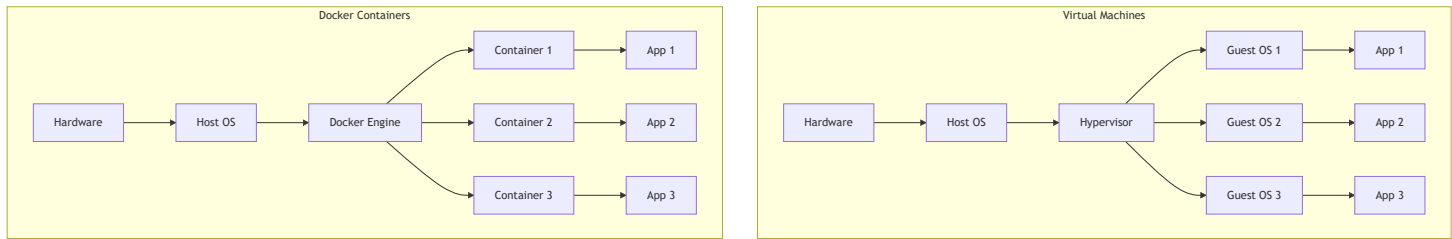
Benefit	Description
Consistency	Applications run the same across all environments
Portability	Run anywhere - development, testing, production
Efficiency	Lightweight compared to traditional VMs
Scalability	Easy to scale up or down
Isolation	Applications don't interfere with each other
Speed	Fast startup times (seconds vs minutes)

Docker Ecosystem



Docker vs Virtual Machines

Architecture Comparison



Detailed Comparison

Aspect	Virtual Machines	Docker Containers
OS	Full guest OS per VM	Shares host OS kernel
Size	GBs	MBs
Startup Time	Minutes	Seconds
Resource Usage	High	Low
Isolation	Hardware-level	Process-level
Portability	Limited	High
Performance	Overhead due to hypervisor	Near-native performance

When to Use What?

Use Virtual Machines When

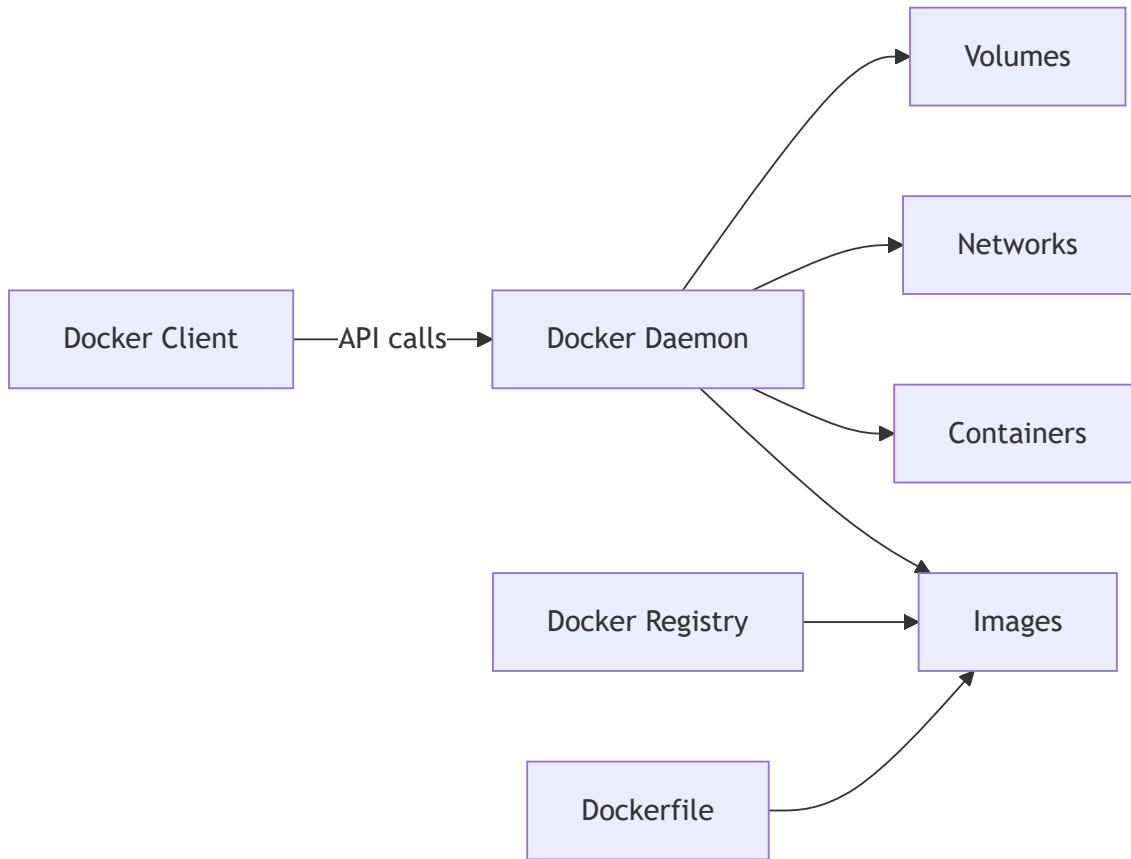
- Need different operating systems
- Require hardware-level isolation
- Running legacy applications
- Strong security requirements

Use Docker Containers When

- Microservices architecture
- CI/CD pipelines
- Development environment consistency
- Rapid scaling needed

Docker Architecture

Core Components



Docker Engine Components

1. Docker Daemon (dockerd)

- Background service managing Docker objects
- Listens for Docker API requests
- Manages images, containers, networks, and volumes

2. Docker Client (docker)

- Command-line interface for users
- Communicates with Docker daemon via REST API
- Can communicate with remote daemons

3. Docker Registry

- Stores Docker images
- Docker Hub is the default public registry
- Private registries available (AWS ECR, Google GCR, etc.)

Docker Objects

Images

- Read-only templates used to create containers
- Built from Dockerfiles
- Layered filesystem

Containers

- Runnable instances of images
- Isolated processes
- Can be started, stopped, moved, and deleted

Networks

- Enable communication between containers
- Bridge, host, overlay network types

Volumes

- Persistent data storage
- Survive container lifecycle

Docker Images

What are Docker Images?

A Docker image is a lightweight, standalone, executable package that includes everything needed to run an application: the code, runtime, system tools, system libraries, and settings.

Image Layers

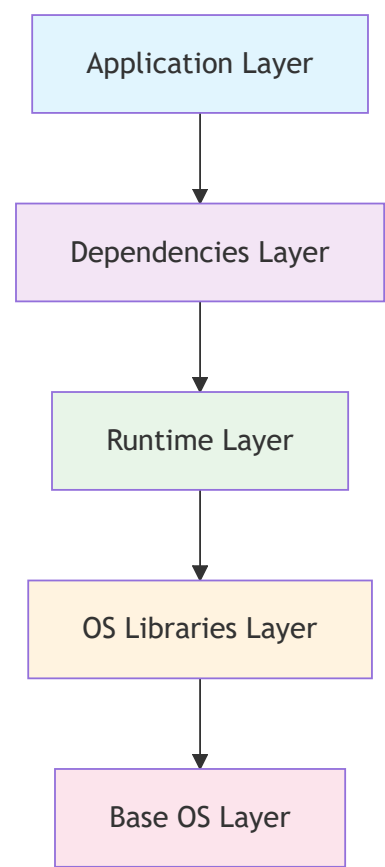


Image Characteristics

Characteristic	Description
Immutable	Once created, cannot be changed
Layered	Built in layers, each layer cached
Portable	Run on any system with Docker
Versioned	Tagged with versions
Lightweight	Share common layers

Image Management Commands

List images

```
docker images
```

Pull image from registry

```
docker pull ubuntu:20.04
```

Build image from Dockerfile

```
docker build -t myapp:v1.0 .
```

Remove image

```
docker rmi myapp:v1.0
```

Tag image

```
docker tag myapp:v1.0 myapp:latest
```

Push to registry

```
docker push myapp:v1.0
```

Inspect image

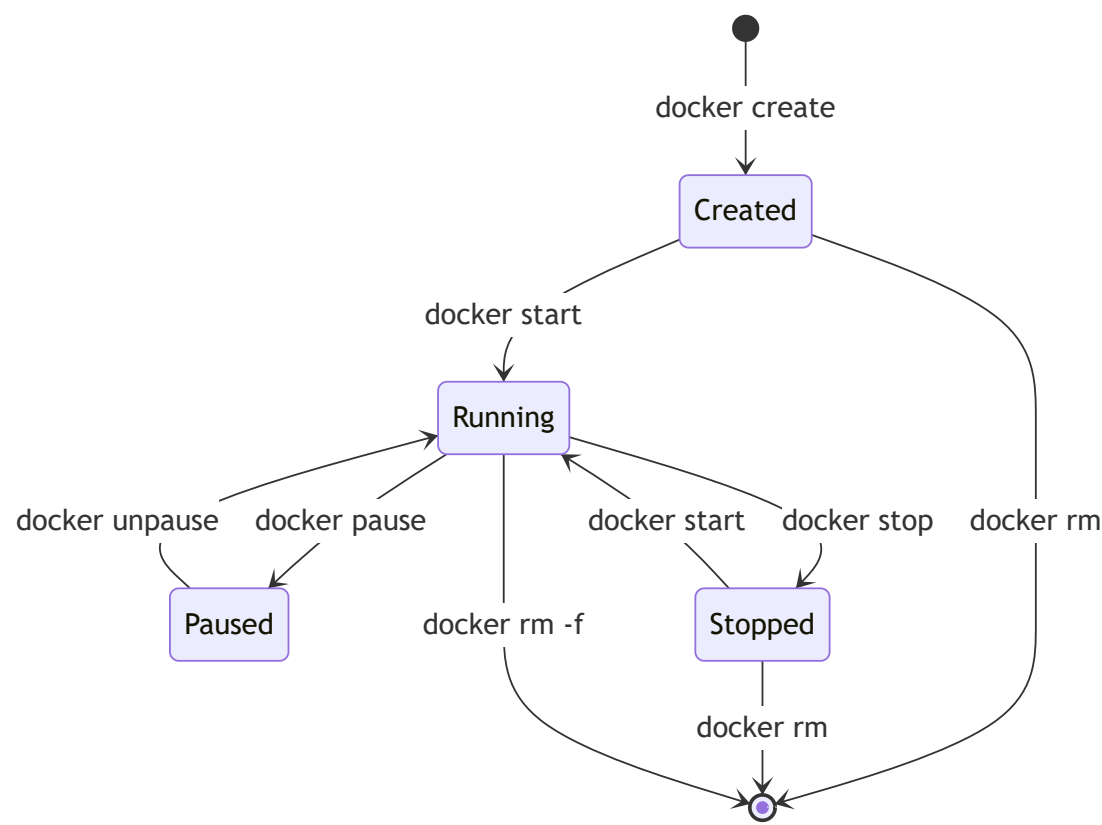
```
docker inspect ubuntu:20.04
```

View image history

```
docker history ubuntu:20.04
```

Docker Containers

Container Lifecycle



Container States

State	Description
Created	Container created but not started
Running	Container is actively running
Paused	Container processes are paused
Restarting	Container is restarting
Exited	Container has stopped
Dead	Container daemon has died

Container Management Commands

```
# Run container (create + start)
docker run -d --name myapp nginx
```

```
# List running containers
docker ps
```

```
# List all containers
docker ps -a
```

```
# Stop container
docker stop myapp
```

```
# Start stopped container
docker start myapp
```

```
# Restart container
docker restart myapp
```

```
# Remove container
docker rm myapp
```

```
# Execute command in running container
docker exec -it myapp bash
```

```
# View container logs
docker logs myapp
```

```
# Inspect container
docker inspect myapp
```

```
# View container resource usage
docker stats myapp
```

Building Docker Images

Dockerfile Fundamentals

A Dockerfile is a text file containing instructions to build a Docker image.

Basic Dockerfile Structure

```
# Base image
FROM node:18-alpine

# Set maintainer
LABEL maintainer="your-email@example.com"

# Set working directory
WORKDIR /app

# Copy package files
COPY package*.json ./

# Install dependencies
RUN npm install --only=production

# Copy application code
COPY . .

# Expose port
EXPOSE 3000

# Set default command
CMD ["npm", "start"]
```

Dockerfile Instructions

Instruction	Purpose	Example
FROM	Base image	FROM ubuntu:20.04
RUN	Execute command	RUN apt-get update
COPY	Copy files	COPY . /app
ADD	Copy files (with URL/archive support)	ADD https://example.com/file.tar.gz /app
WORKDIR	Set working directory	WORKDIR /app
EXPOSE	Document port	EXPOSE 8080
ENV	Set environment variable	ENV NODE_ENV=production
CMD	Default command	CMD ["python", "app.py"]
ENTRYPOINT	Container entry point	ENTRYPOINT ["python"]
VOLUME	Mount point	VOLUME ["/data"]

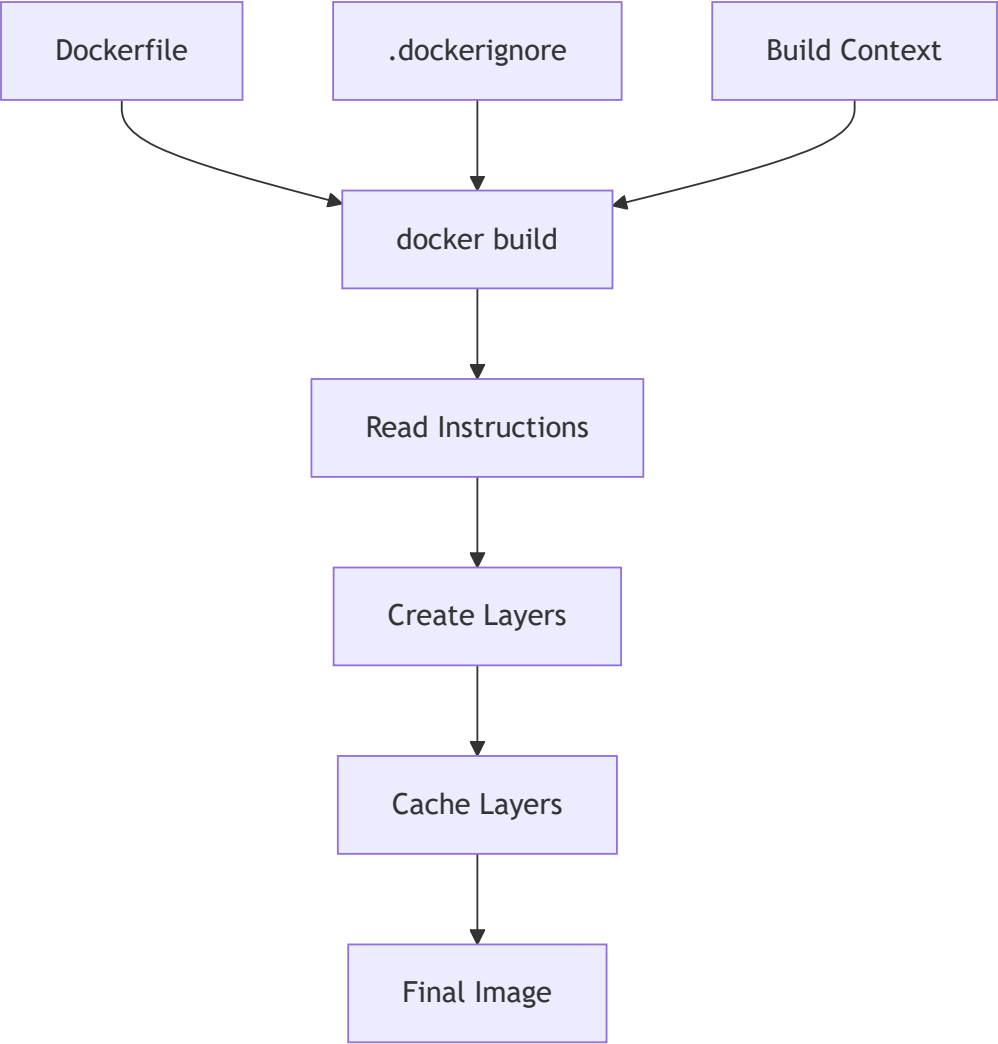
Instruction	Purpose	Example
USER	Set user	USER 1000
ARG	Build argument	ARG VERSION=latest

Multi-Stage Builds

```
# Build stage
FROM node:18 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# Production stage
FROM nginx:alpine
COPY --from=builder /app/build /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Build Process



Build Commands

Basic build

```
docker build -t myapp .
```

Build with tag

```
docker build -t myapp:v1.0 .
```

Build with custom Dockerfile

```
docker build -f Dockerfile.prod -t myapp:prod .
```

Build with build arguments

```
docker build --build-arg VERSION=1.0 -t myapp .
```

Build without cache

```
docker build --no-cache -t myapp .
```

Build and view progress

```
docker build --progress=plain -t myapp .
```

Running Docker Containers

Basic Container Operations

```
# Run container in foreground
docker run ubuntu echo "Hello World"

# Run container in background
docker run -d nginx

# Run interactive container
docker run -it ubuntu bash

# Run with custom name
docker run --name myapp nginx

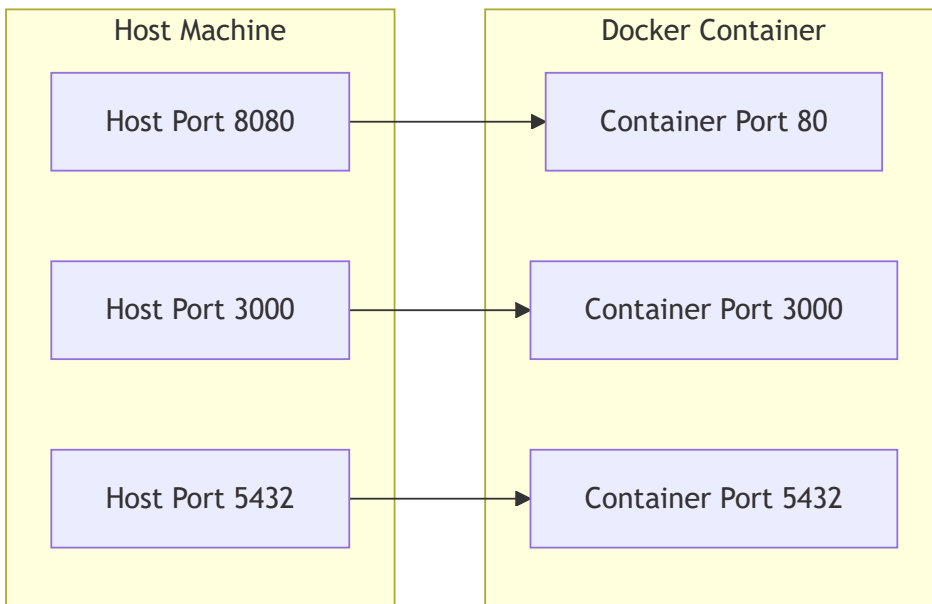
# Run with port mapping
docker run -p 8080:80 nginx

# Run with volume mounting
docker run -v /host/path:/container/path nginx

# Run with environment variables
docker run -e ENV_VAR=value nginx

# Run with resource limits
docker run --memory=512m --cpus=1 nginx
```

Port Mapping



Volume Types

1. Bind Mounts

```
docker run -v /host/path:/container/path nginx
```

2. Named Volumes

```
docker volume create myvolume
docker run -v myvolume:/data nginx
```

3. Anonymous Volumes

```
docker run -v /data nginx
```

Networking

Network Types

Type	Description	Use Case
Bridge	Default network	Single host communication
Host	Use host networking	Performance-critical apps
None	No networking	Isolated containers
Overlay	Multi-host networking	Docker Swarm

Network Commands

```
# List networks
docker network ls

# Create network
docker network create mynetwork

# Run container on specific network
docker run --network mynetwork nginx

# Connect container to network
docker network connect mynetwork mycontainer

# Inspect network
docker network inspect mynetwork
```

Docker Commands Reference

Essential Commands Cheatsheet

System Commands

```
# Version and info
docker --version
docker version
docker info
docker system info

# Help
docker --help
docker <command> --help
```

Image Commands

```
# List images
docker images
docker image ls

# Pull image
docker pull <image>:<tag>

# Build image
docker build -t <name>:<tag> .

# Remove image
docker rmi <image>

# Remove unused images
docker image prune

# Save image to file
docker save -o image.tar <image>

# Load image from file
docker load -i image.tar
```

Container Commands

Run container

```
docker run [options] <image> [command]
```

List containers

```
docker ps                # Running containers
```

```
docker ps -a            # All containers
```

Container lifecycle

```
docker start <container>
```

```
docker stop <container>
```

```
docker restart <container>
```

```
docker pause <container>
```

```
docker unpause <container>
```

Remove container

```
docker rm <container>
```

```
docker rm -f <container> # Force remove
```

Execute command

```
docker exec -it <container> <command>
```

View logs

```
docker logs <container>
```

```
docker logs -f <container> # Follow logs
```

Copy files

```
docker cp <container>:/path /host/path
```

```
docker cp /host/path <container>:/path
```

Volume Commands

```
# List volumes
docker volume ls

# Create volume
docker volume create <volume>

# Remove volume
docker volume rm <volume>

# Inspect volume
docker volume inspect <volume>

# Remove unused volumes
docker volume prune
```

Volume Commands

```
# List volumes
docker volume ls

# Create volume
docker volume create <volume>

# Remove volume
docker volume rm <volume>

# Inspect volume
docker volume inspect <volume>

# Remove unused volumes
docker volume prune
```

Network Commands

```
# List networks
docker network ls

# Create network
docker network create <network>

# Remove network
docker network rm <network>

# Inspect network
docker network inspect <network>

# Connect container to network
docker network connect <network> <container>

# Disconnect container from network
docker network disconnect <network> <container>

# Remove unused networks
docker network prune
```

Docker Volumes and Data Management

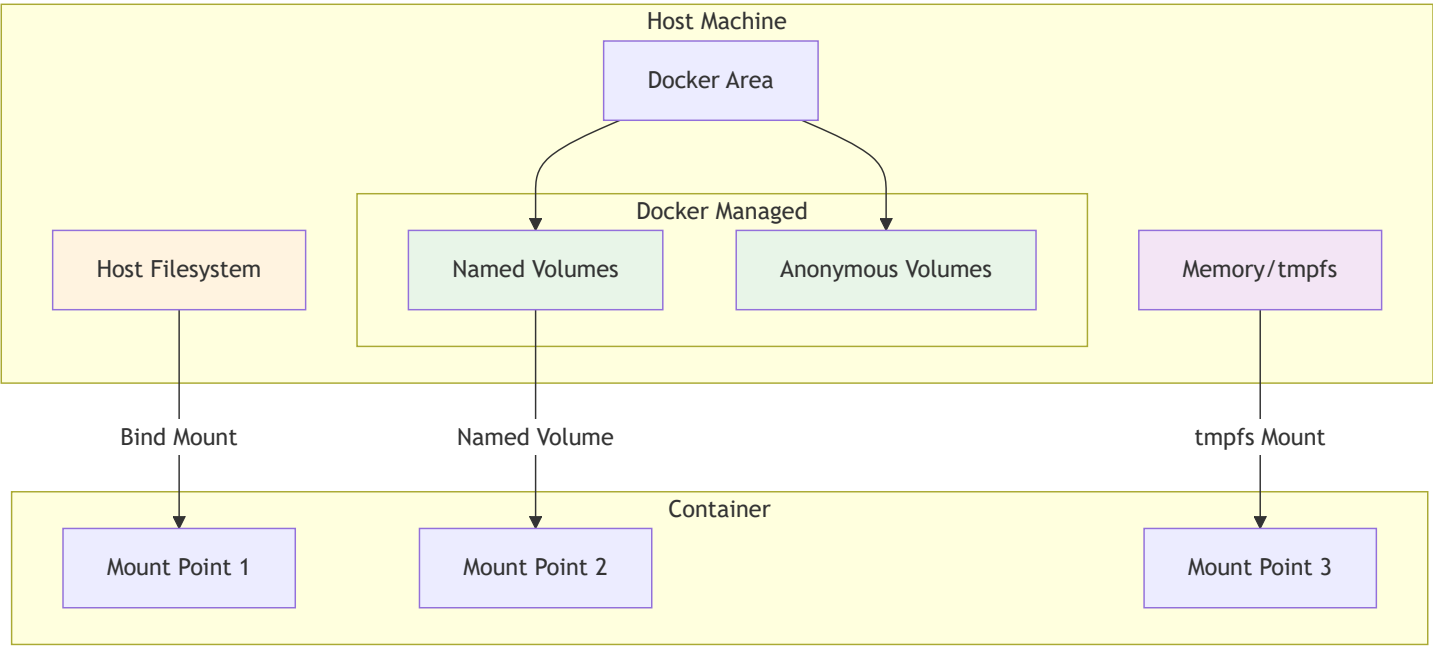
Understanding Data Persistence in Docker

By default, all files created inside a container are stored on a writable container layer. This means that:

- Data doesn't persist when the container no longer exists
- It's difficult to get data out of the container
- Writing into a container's writable layer requires a storage driver to manage the filesystem

Docker provides three ways to mount data into a container: **volumes**, **bind mounts**, and **tmpfs mounts**.

Volume Types Comparison



Volume Types Detailed

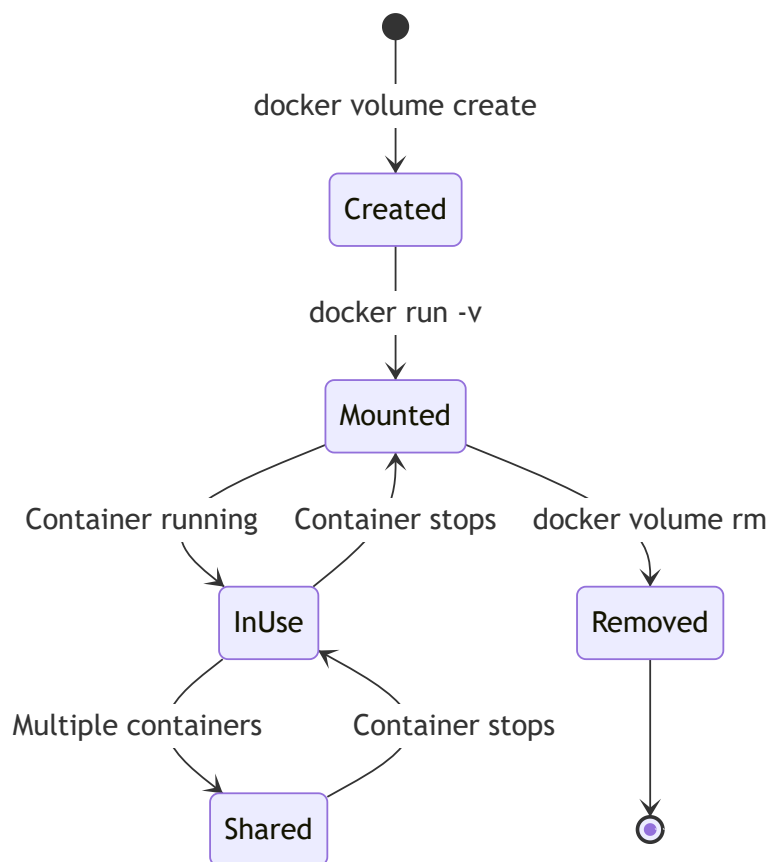
Type	Storage Location	Managed By	Persistence	Performance	Use Case
Named Volumes	Docker area (/var/lib/docker/volumes/)	Docker	Yes	High	Database data, application data
Bind Mounts	Anywhere on host	User	Yes	Variable	Development, config files
Anonymous Volumes	Docker area	Docker	Container lifetime	High	Temporary data, cache
tmpfs Mounts	Host memory	Docker	No	Very High	Sensitive data, temporary files

Docker Volumes (Named Volumes)

What are Named Volumes?

Named volumes are the preferred mechanism for persisting data generated by and used by Docker containers. They are completely managed by Docker.

Volume Lifecycle



Volume Management Commands

```
# Create a named volume
docker volume create myvolume

# List all volumes
docker volume ls

# Inspect volume details
docker volume inspect myvolume

# Remove a volume
docker volume rm myvolume

# Remove all unused volumes
docker volume prune

# Remove all volumes (dangerous!)
docker volume prune -a
```

Using Named Volumes

```
# Create and use a volume in one command
docker run -d --name db -v postgres_data:/var/lib/postgresql/data postgres:13

# Create volume first, then use it
docker volume create app_data
docker run -d --name app -v app_data:/app/data nginx

# Share volume between containers
docker run -d --name writer -v shared_data:/data alpine sh -c "echo 'Hello' > /data/message.txt"
docker run --rm -v shared_data:/data alpine cat /data/message.txt
```

Volume Inspection Example

```
$ docker volume inspect postgres_data
[
  {
    "CreatedAt": "2025-01-15T10:30:00Z",
    "Driver": "local",
    "Labels": {},
    "Mountpoint": "/var/lib/docker/volumes/postgres_data/_data",
    "Name": "postgres_data",
    "Options": {},
    "Scope": "local"
  }
]
```

Bind Mounts

What are Bind Mounts?

Bind mounts have been around since the early days of Docker. When you use a bind mount, a file or directory on the host machine is mounted into a container.

Bind Mount Characteristics

Aspect	Description
Path	Absolute path on host required
Performance	Direct access to host filesystem
Permissions	Uses host file permissions
Backup	Manual backup required

Aspect	Description
Portability	Host-dependent paths

Bind Mount Syntax

```
# Using -v flag (older syntax)
docker run -v /host/path:/container/path image

# Using --mount flag (recommended)
docker run --mount type=bind,source=/host/path,target=/container/path image

# Read-only bind mount
docker run --mount type=bind,source=/host/path,target=/container/path,readonly image
```

Bind Mount Examples

```
# Development environment - sync source code
docker run -d \
  --name dev-app \
  --mount type=bind,source="$(pwd)",target=/app \
  node:18 \
  npm run dev

# Configuration file mounting
docker run -d \
  --name nginx \
  --mount type=bind,source=/host/nginx.conf,target=/etc/nginx/nginx.conf,readonly \
  -p 80:80 \
  nginx

# Log file access
docker run -d \
  --name app \
  --mount type=bind,source=/var/log/myapp,target=/app/logs \
  myapp:latest
```

Anonymous Volumes

Anonymous volumes are created when you specify a volume mount point without a name.

```
# Anonymous volume
docker run -v /data nginx

# Inspect anonymous volumes
docker volume ls

# DRIVER      VOLUME NAME
# local       a1b2c3d4e5f6...
```

tmpfs Mounts

tmpfs mounts are stored in the host system's memory only, never written to the host filesystem.

```
# tmpfs mount for sensitive data
docker run -d \
  --name secure-app \
  --tmpfs /tmp:rw,noexec,nosuid,size=100m \
  myapp:latest

# Using --mount syntax
docker run -d \
  --name secure-app \
  --mount type=tmpfs,destination=/tmp,tmpfs-size=100m \
  myapp:latest
```

Volume Use Cases and Examples

Database Persistence

```
# PostgreSQL with named volume
docker run -d \
  --name postgres \
  -e POSTGRES_DB=myapp \
  -e POSTGRES_USER=user \
  -e POSTGRES_PASSWORD=password \
  -v postgres_data:/var/lib/postgresql/data \
  postgres:13

# MySQL with named volume
docker run -d \
  --name mysql \
  -e MYSQL_ROOT_PASSWORD=rootpass \
  -e MYSQL_DATABASE=myapp \
  -v mysql_data:/var/lib/mysql \
  mysql:8.0
```

Development Environment

```
# Node.js development with bind mount
docker run -d \
  --name node-dev \
  --mount type=bind,source="$(pwd)",target=/app \
  --mount type=volume,source=node_modules,target=/app/node_modules \
  -p 3000:3000 \
  -w /app \
  node:18 \
  npm run dev
```

Configuration Management

```
# Nginx with configuration bind mount
docker run -d \
  --name nginx \
  -p 80:80 \
  --mount type=bind,source=/host/nginx.conf,target=/etc/nginx/nginx.conf,readonly \
  --mount type=bind,source=/host/ssl,target=/etc/nginx/ssl,readonly \
  nginx:alpine
```

Volume Best Practices

1. Use Named Volumes for Data Persistence

```
# Good: Named volume for database
docker run -v db_data:/var/lib/mysql mysql

# Avoid: Anonymous volume for important data
docker run -v /var/lib/mysql mysql
```

2. Use Bind Mounts for Development

```
# Development: Bind mount source code
docker run --mount type=bind,source="$(pwd)",target=/app node:18

# Production: Copy code into image
# In Dockerfile: COPY . /app
```

3. Backup Volumes Regularly

```
# Backup named volume
docker run --rm \
  -v myvolume:/data \
  -v $(pwd):/backup \
  alpine tar czf /backup/myvolume-backup.tar.gz -C /data .

# Restore named volume
docker run --rm \
  -v myvolume:/data \
  -v $(pwd):/backup \
  alpine tar xzf /backup/myvolume-backup.tar.gz -C /data
```

4. Set Proper Permissions

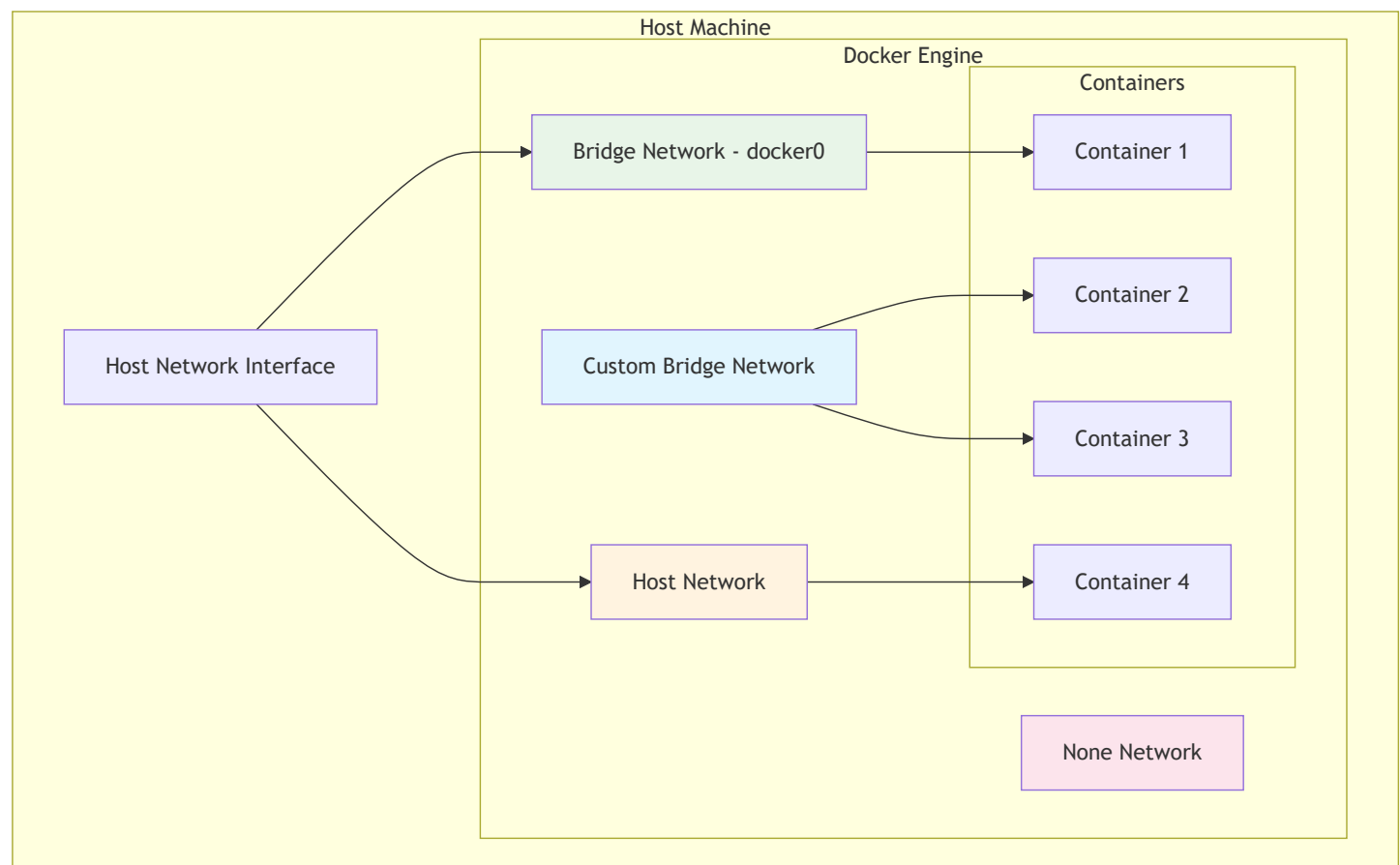
```
# Run container with specific user
docker run --user $(id -u):$(id -g) \
  --mount type=bind,source="$(pwd)",target=/app \
  node:18
```

Docker Networks

Understanding Docker Networking

Docker networking allows containers to communicate with each other and with external networks. By default, Docker provides several network drivers to support different networking scenarios.

Network Architecture Overview



Network Drivers

Driver	Description	Use Case	Scope
bridge	Default network driver	Single host networking	Local
host	Remove network isolation	High performance	Local
overlay	Connect multiple Docker hosts	Swarm services	Swarm
ipvlan	Control over IPv4/IPv6 addressing	Legacy applications	Local
macvlan	Assign MAC address to container	Legacy applications	Local
none	Disable networking	Maximum isolation	Local

Default Bridge Network

When Docker is installed, it creates a default bridge network called `bridge`.

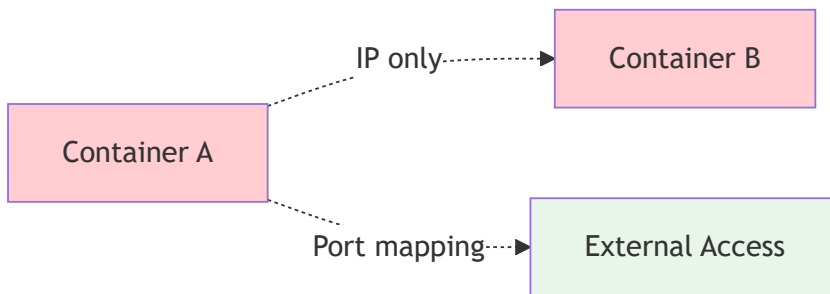
```
# List networks
docker network ls

# Inspect default bridge
docker network inspect bridge

# Containers on default bridge
docker run -d --name web1 nginx
docker run -d --name web2 nginx

# Check container IP addresses
docker inspect web1 | grep IPAddress
docker inspect web2 | grep IPAddress
```

Default Bridge Network Limitations



- Containers can only communicate via IP addresses
- No automatic DNS resolution between containers
- All containers share the same network namespace

User-Defined Bridge Networks

User-defined bridge networks provide better isolation and features compared to the default bridge.

Creating Custom Networks

```
# Create a custom bridge network
docker network create mynetwork

# Create network with custom settings
docker network create \
  --driver bridge \
  --subnet=172.20.0.0/16 \
  --ip-range=172.20.240.0/20 \
  --gateway=172.20.0.1 \
  mynetwork

# Create network with custom options
docker network create \
  --driver bridge \
  --opt com.docker.network.bridge.name=mybr0 \
  --opt com.docker.network.driver.mtu=1200 \
  mynetwork
```

Using Custom Networks

```
# Run containers on custom network
docker run -d --name web --network mynetwork nginx
docker run -d --name db --network mynetwork postgres:13

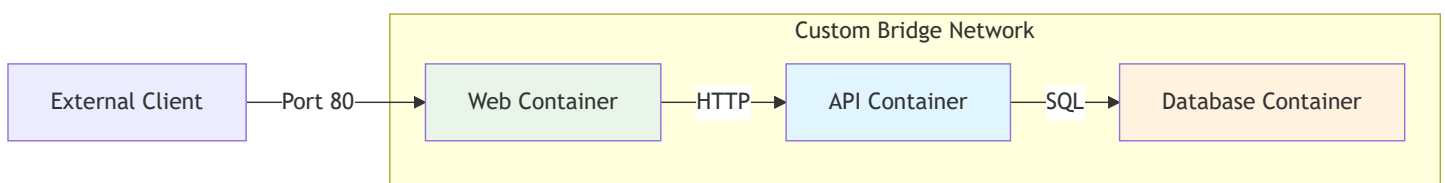
# Containers can communicate by name
docker exec web ping db # This works!

# Connect existing container to network
docker network connect mynetwork existing_container

# Disconnect container from network
docker network disconnect mynetwork existing_container
```

Network Communication Examples

Same Network Communication



```
# Create network
docker network create app-network

# Run database
docker run -d \
  --name postgres \
  --network app-network \
  -e POSTGRES_DB=myapp \
  postgres:13

# Run API (connects to database by name)
docker run -d \
  --name api \
  --network app-network \
  -e DATABASE_URL=postgres://postgres@postgres:5432/myapp \
  myapi:latest

# Run web frontend
docker run -d \
  --name web \
  --network app-network \
  -p 80:80 \
  nginx
```

Multi-Network Setup

```
# Create networks
docker network create frontend
docker network create backend

# Database only on backend
docker run -d --name db --network backend postgres:13

# API on both networks
docker run -d --name api myapi:latest
docker network connect frontend api
docker network connect backend api

# Web only on frontend
docker run -d --name web --network frontend -p 80:80 nginx
```

Host Network

Containers using host networking share the host's network stack directly.

```
# Run container with host networking
docker run -d --network host nginx

# Container ports are directly available on host
curl localhost:80 # Direct access without port mapping
```

Host Network Pros and Cons

Pros	Cons
Best performance	No isolation
No port mapping needed	Port conflicts possible
Access to host interfaces	Security concerns

Network Inspection and Debugging

Network Commands

```
# List networks
docker network ls

# Inspect network details
docker network inspect mynetwork

# Remove network
docker network rm mynetwork

# Remove unused networks
docker network prune
```

Container Network Information

```
# Inspect container network settings
docker inspect container_name | grep -A 20 NetworkSettings

# Check container connectivity
docker exec container_name ping google.com
docker exec container_name nslookup google.com

# View container network interfaces
docker exec container_name ip addr show

# Check listening ports
docker exec container_name netstat -tulpn
```

Advanced Networking Scenarios

Load Balancing with Multiple Containers

```
# Create network
docker network create lb-network

# Run multiple web servers
docker run -d --name web1 --network lb-network nginx
docker run -d --name web2 --network lb-network nginx
docker run -d --name web3 --network lb-network nginx

# Run load balancer
docker run -d --name lb \
  --network lb-network \
  -p 80:80 \
  -v $(pwd)/nginx.conf:/etc/nginx/nginx.conf \
  nginx
```

Service Discovery Example

```
# Create network with DNS
docker network create --driver bridge app-net

# Run services
docker run -d --name redis --network app-net redis:alpine
docker run -d --name postgres --network app-net postgres:13

# Application can connect using service names
docker run -d --name app \
  --network app-net \
  -e REDIS_URL=redis://redis:6379 \
  -e DATABASE_URL=postgres://postgres:5432/myapp \
  myapp:latest
```

Network Security Best Practices

1. Use Custom Networks

```
# Good: Custom network with isolation
docker network create secure-network
docker run --network secure-network myapp

# Avoid: Default bridge for production
docker run myapp # Uses default bridge
```

2. Implement Network Segmentation

```
# Separate networks for different tiers
docker network create frontend-net
docker network create backend-net
docker network create db-net

# Web tier
docker run --network frontend-net web-app

# API tier (connects to both frontend and backend)
docker network connect frontend-net api-app
docker network connect backend-net api-app

# Database tier
docker run --network db-net database
```

3. Use Firewall Rules

```
# Limit external access
iptables -A DOCKER-USER -i eth0 -p tcp --dport 22 -j DROP

# Allow specific networks only
iptables -A DOCKER-USER -s 192.168.1.0/24 -j ACCEPT
```

Docker Network with Docker Compose

Docker Compose automatically creates a network for your application.

```
version: '3.8'

services:
  web:
    image: nginx
    networks:
      - frontend
      - backend

  api:
    image: myapi:latest
    networks:
      - backend
      - database

  db:
    image: postgres:13
    networks:
      - database

networks:
  frontend:
    driver: bridge
  backend:
    driver: bridge
  database:
    driver: bridge
    internal: true # No external access
```

Network Troubleshooting

Common Issues and Solutions

```
# Issue: Containers can't communicate
# Solution: Check if they're on the same network
docker network inspect network_name

# Issue: DNS resolution not working
# Solution: Use custom networks (not default bridge)
docker network create mynet
docker run --network mynet --name app1 image1
docker run --network mynet --name app2 image2

# Issue: Port conflicts
# Solution: Use different host ports
docker run -p 8081:80 nginx # Instead of 8080:80

# Issue: Network performance
# Solution: Use host networking for high performance
docker run --network host myapp
```

Network Monitoring

```
# Monitor network traffic
docker exec container_name ss -tuln
docker exec container_name iftop
docker exec container_name tcpdump -i eth0

# Check connectivity between containers
docker exec container1 ping container2
docker exec container1 telnet container2 5432
```

Docker Compose

What is Docker Compose?

Compose is a tool for defining and running multi-container Docker applications. With Compose, you use a YAML file to configure your application's services. Then, with a single command, you create and start all the services from your configuration.

docker-compose.yml Structure

```
version: '3.8'

services:
  web:
    build: .
    ports:
      - "3000:3000"
    environment:
      - NODE_ENV=development
    volumes:
      - ./app
      - /app/node_modules
    depends_on:
      - db
      - redis

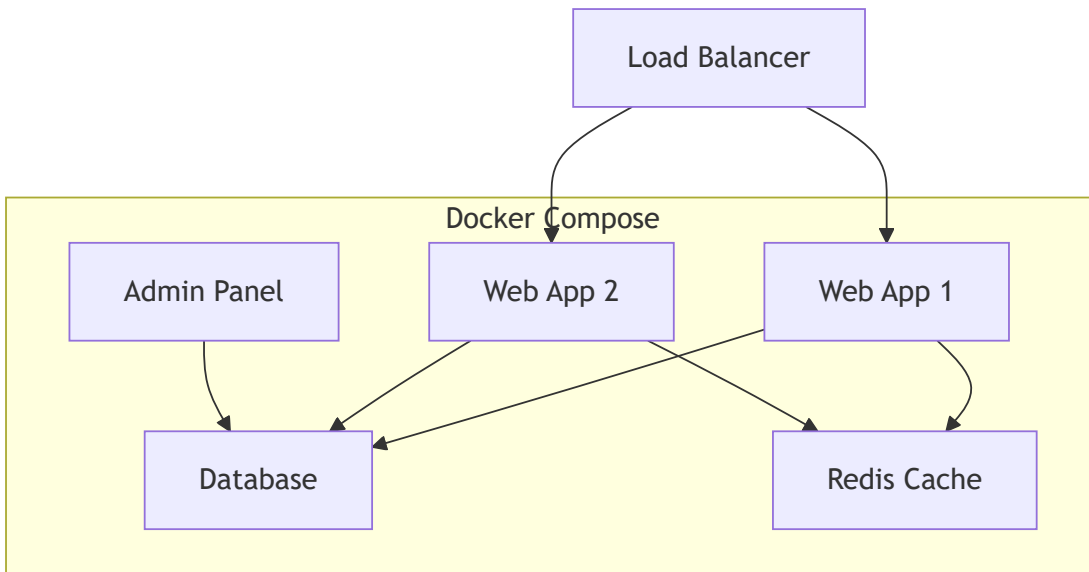
  db:
    image: postgres:13
    environment:
      POSTGRES_DB: myapp
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
      - "5432:5432"

  redis:
    image: redis:alpine
    ports:
      - "6379:6379"

volumes:
  postgres_data:

networks:
  default:
    driver: bridge
```

Multi-Container Application Architecture



Compose Commands

```
# Start services
docker-compose up

# Start in background
docker-compose up -d

# Build and start
docker-compose up --build

# Stop services
docker-compose down

# Stop and remove volumes
docker-compose down --volumes

# View logs
docker-compose logs

# Scale services
docker-compose up --scale web=3

# Execute command
docker-compose exec web bash

# View service status
docker-compose ps
```

Best Practices

Dockerfile Best Practices

1. Use Multi-Stage Builds

```
# Build stage
FROM node:18 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production

# Production stage
FROM node:18-alpine
WORKDIR /app
COPY --from=builder /app/node_modules ./node_modules
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
```

2. Minimize Layers

```
# Bad - Multiple RUN commands
RUN apt-get update
RUN apt-get install -y curl
RUN apt-get install -y wget

# Good - Single RUN command
RUN apt-get update && \
    apt-get install -y curl wget && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/*
```

3. Use Specific Tags

```
# Bad
FROM node

# Good
FROM node:18-alpine
```

4. Leverage Build Cache

```
# Copy package files first
COPY package*.json ./
RUN npm install

# Copy source code last
COPY . .
```

Security Best Practices

1. Run as Non-Root User

```
RUN groupadd -r app && useradd -r -g app app
USER app
```

2. Use Official Images

```
FROM node:18-alpine # Official Node.js image
```

3. Scan for Vulnerabilities

```
docker scan myapp:latest
```

4. Keep Images Updated

```
# Regularly update base images
docker pull node:18-alpine
docker build --no-cache -t myapp .
```

Performance Best Practices

1. Use .dockerignore

```
node_modules
npm-debug.log
.git
.gitignore
README.md
.env
.nyc_output
coverage
.tmp
```

2. Optimize Image Size

```
# Use Alpine images
FROM node:18-alpine

# Clean up in same layer
RUN apt-get update && \
    apt-get install -y package && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/*
```

3. Use Health Checks

```
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
    CMD curl -f http://localhost:3000/health || exit 1
```

Development Best Practices

1. Use Environment Variables

```
ENV NODE_ENV=production
ENV PORT=3000
```

2. Document Your Images

```
LABEL maintainer="developer@company.com"
LABEL version="1.0"
LABEL description="My application"
```

3. Use Proper Resource Limits

```
docker run --memory=512m --cpus=1 myapp
```

Hands-on Examples

Example 1: Simple Web Application

Dockerfile

```
FROM nginx:alpine

# Copy custom configuration
COPY nginx.conf /etc/nginx/nginx.conf

# Copy static files
COPY html/ /usr/share/nginx/html/

# Expose port
EXPOSE 80

# Health check
HEALTHCHECK --interval=30s --timeout=3s \
  CMD wget --quiet --tries=1 --spider http://localhost:80/ || exit 1
```

Build and Run

```
# Build image
docker build -t my-web-app .

# Run container
docker run -d -p 8080:80 --name webapp my-web-app

# Test
curl http://localhost:8080
```

Example 2: Node.js Application with Database

Project Structure

```
myapp/  
├─ Dockerfile  
├─ docker-compose.yml  
├─ package.json  
├─ app.js  
└─ .dockerignore
```

Dockerfile

```
FROM node:18-alpine  
  
WORKDIR /app  
  
# Copy package files  
COPY package*.json ./  
  
# Install dependencies  
RUN npm ci --only=production  
  
# Copy application code  
COPY . .  
  
# Create non-root user  
RUN addgroup -g 1001 -S nodejs  
RUN adduser -S nextjs -u 1001  
USER nextjs  
  
EXPOSE 3000  
  
CMD ["npm", "start"]
```

docker-compose.yml

```
version: '3.8'

services:
  app:
    build: .
    ports:
      - "3000:3000"
    environment:
      - DATABASE_URL=postgres://user:pass@db:5432/myapp
      - REDIS_URL=redis://redis:6379
    depends_on:
      - db
      - redis
    restart: unless-stopped

  db:
    image: postgres:13-alpine
    environment:
      POSTGRES_DB: myapp
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
    volumes:
      - postgres_data:/var/lib/postgresql/data
    restart: unless-stopped

  redis:
    image: redis:alpine
    restart: unless-stopped

volumes:
  postgres_data:
```

Commands

```
# Start the application
docker-compose up -d

# View logs
docker-compose logs -f app

# Scale the application
docker-compose up -d --scale app=3

# Stop everything
docker-compose down
```

Example 4: Full-Stack Application with Volumes and Networks

Project Structure

```
fullstack-app/
├─ docker-compose.yml
├─ web/
│   └─ Dockerfile
│   └─ index.html
├─ api/
│   └─ Dockerfile
│   └─ package.json
│   └─ server.js
└─ database/
    └─ init.sql
```

docker-compose.yml with Networks and Volumes

```
version: '3.8'

services:
  # Frontend Web Server
  web:
    build: ./web
    ports:
      - "80:80"
    networks:
      - frontend
    volumes:
      - web_logs:/var/log/nginx
    depends_on:
      - api
    restart: unless-stopped

  # Backend API
  api:
    build: ./api
    ports:
      - "3000:3000"
    networks:
      - frontend
      - backend
    environment:
      - DATABASE_URL=postgres://user:pass@db:5432/myapp
      - REDIS_URL=redis://cache:6379
    volumes:
      - api_uploads:/app/uploads
      - ./api/logs:/app/logs # Bind mount for development
    depends_on:
      - db
      - cache
    restart: unless-stopped

  # Database
  db:
    image: postgres:13-alpine
    networks:
      - backend
    environment:
      POSTGRES_DB: myapp
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
    volumes:
      - postgres_data:/var/lib/postgresql/data
```

```
- ./database/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
restart: unless-stopped
```

```
# Redis Cache
```

```
cache:
```

```
  image: redis:alpine
```

```
  networks:
```

```
    - backend
```

```
  volumes:
```

```
    - redis_data:/data
```

```
  command: redis-server --appendonly yes
```

```
  restart: unless-stopped
```

```
# Database Admin (accessible from frontend network)
```

```
adminer:
```

```
  image: adminer
```

```
  networks:
```

```
    - frontend
```

```
    - backend
```

```
  ports:
```

```
    - "8080:8080"
```

```
  depends_on:
```

```
    - db
```

```
volumes:
```

```
  postgres_data:
```

```
  redis_data:
```

```
  web_logs:
```

```
  api_uploads:
```

```
networks:
```

```
  frontend:
```

```
    driver: bridge
```

```
  backend:
```

```
    driver: bridge
```

```
    internal: true # Backend network has no external access
```

Commands and Testing

Start the entire stack

```
docker-compose up -d
```

Check networks created

```
docker network ls | grep fullstack
```

Inspect backend network (should be internal)

```
docker network inspect fullstack-app_backend
```

Test connectivity between services

```
docker-compose exec api ping db
```

```
docker-compose exec api ping cache
```

Check volume usage

```
docker volume ls | grep fullstack
```

```
docker system df # View space usage
```

View logs from specific service

```
docker-compose logs -f api
```

Scale API service

```
docker-compose up -d --scale api=3
```

Backup database volume

```
docker run --rm \
  -v fullstack-app_postgres_data:/data \
  -v $(pwd)/backup:/backup \
  alpine tar czf /backup/db-backup.tar.gz -C /data .
```

Restore database volume

```
docker run --rm \
  -v fullstack-app_postgres_data:/data \
  -v $(pwd)/backup:/backup \
  alpine tar xzf /backup/db-backup.tar.gz -C /data
```

Example 5: Development Environment with Live Reload

Development Setup with Bind Mounts

```
version: '3.8'

services:
  # Frontend Development Server
  web-dev:
    image: node:18-alpine
    working_dir: /app
    networks:
      - dev-network
    ports:
      - "3000:3000"
    volumes:
      # Bind mount source code for live reload
      - ./frontend:/app
      # Named volume for node_modules (performance)
      - node_modules:/app/node_modules
    command: npm run dev
    environment:
      - CHOKIDAR_USEPOLLING=true # For file watching on all platforms

  # Backend Development Server
  api-dev:
    image: node:18-alpine
    working_dir: /app
    networks:
      - dev-network
    ports:
      - "3001:3001"
    volumes:
      - ./backend:/app
      - api_node_modules:/app/node_modules
    command: npm run dev
    environment:
      - NODE_ENV=development
      - DATABASE_URL=postgres://dev:dev@db:5432/devdb

  # Development Database
  db:
    image: postgres:13-alpine
    networks:
      - dev-network
    environment:
      POSTGRES_DB: devdb
```

```
POSTGRES_USER: dev
POSTGRES_PASSWORD: dev
volumes:
  - dev_postgres:/var/lib/postgresql/data
  # Mount init scripts
  - ./database/dev-data.sql:/docker-entrypoint-initdb.d/init.sql:ro
ports:
  - "5432:5432" # Expose for external tools
```

```
volumes:
  node_modules:
  api_node_modules:
  dev_postgres:
```

```
networks:
  dev-network:
    driver: bridge
```

Example 6: Microservices with Service Discovery

```
version: '3.8'

services:
  # API Gateway
  gateway:
    build: ./gateway
    ports:
      - "80:80"
    networks:
      - public
      - services
    environment:
      - USER_SERVICE_URL=http://user-service:3000
      - ORDER_SERVICE_URL=http://order-service:3000
      - PAYMENT_SERVICE_URL=http://payment-service:3000

  # User Service
  user-service:
    build: ./user-service
    networks:
      - services
      - database
    environment:
      - DATABASE_URL=postgres://user:pass@user-db:5432/users
    volumes:
      - user_uploads:/app/uploads

  # Order Service
  order-service:
    build: ./order-service
    networks:
      - services
      - database
    environment:
      - DATABASE_URL=postgres://user:pass@order-db:5432/orders
      - MESSAGE_QUEUE_URL=redis://message-queue:6379

  # Payment Service
  payment-service:
    build: ./payment-service
    networks:
      - services
      - database
    environment:
      - DATABASE_URL=postgres://user:pass@payment-db:5432/payments
```

Databases (each service has its own)

user-db:

image: postgres:13-alpine

networks:

- database

environment:

POSTGRES_DB: users

POSTGRES_USER: user

POSTGRES_PASSWORD: pass

volumes:

- user_db_data:/var/lib/postgresql/data

order-db:

image: postgres:13-alpine

networks:

- database

environment:

POSTGRES_DB: orders

POSTGRES_USER: user

POSTGRES_PASSWORD: pass

volumes:

- order_db_data:/var/lib/postgresql/data

payment-db:

image: postgres:13-alpine

networks:

- database

environment:

POSTGRES_DB: payments

POSTGRES_USER: user

POSTGRES_PASSWORD: pass

volumes:

- payment_db_data:/var/lib/postgresql/data

Message Queue

message-queue:

image: redis:alpine

networks:

- services

volumes:

- message_queue_data:/data

volumes:

user_uploads:

user_db_data:

order_db_data:

payment_db_data:

```
message_queue_data:

networks:
  public:
    driver: bridge
  services:
    driver: bridge
  database:
    driver: bridge
    internal: true # Database network is isolated
```

Troubleshooting

Common Issues and Solutions

Issue 1: Container Exits Immediately

```
# Check logs
docker logs <container>

# Run interactively to debug
docker run -it <image> /bin/bash

# Check if main process exists
docker run -it <image> ps aux
```

Issue 2: Port Already in Use

```
# Check what's using the port
netstat -tulpn | grep :8080

# Use different port
docker run -p 8081:80 nginx

# Stop conflicting container
docker stop $(docker ps -q --filter "publish=8080")
```

Issue 3: Out of Disk Space

```
# Check Docker disk usage
docker system df

# Clean up
docker system prune -a --volumes

# Remove specific items
docker container prune
docker image prune -a
docker volume prune
```

Issue 4: Container Can't Connect to Network

```
# Inspect network
docker network inspect bridge

# Check container network settings
docker inspect <container> | grep -A 20 NetworkSettings

# Create custom network
docker network create mynetwork
docker run --network mynetwork <image>
```

Issue 6: Volume Mount Problems

```
# Check volume exists
docker volume ls | grep myvolume

# Inspect volume details
docker volume inspect myvolume

# Check mount points
docker inspect container | grep -A 10 Mounts

# Fix permission issues
docker run --user $(id -u):$(id -g) -v /host:/container image

# Debug volume content
docker run --rm -v myvolume:/data alpine ls -la /data
```

Issue 7: Network Connectivity Problems

```
# Check container networks
docker inspect container | grep -A 20 NetworkSettings

# Test connectivity
docker exec container1 ping container2
docker exec container1 nslookup container2

# Check if containers are on same network
docker network inspect network_name

# Create custom network for container communication
docker network create mynetwork
docker run --network mynetwork container1
docker run --network mynetwork container2
```

Debugging Commands

```
# Container inspection
docker inspect <container>
docker logs <container>
docker stats <container>

# Execute commands in container
docker exec -it <container> bash
docker exec -it <container> sh # For Alpine images

# File system inspection
docker exec -it <container> ls -la /
docker exec -it <container> cat /etc/hosts

# Process inspection
docker exec -it <container> ps aux
docker exec -it <container> top

# Network debugging
docker exec -it <container> ping google.com
docker exec -it <container> nslookup google.com
docker exec -it <container> netstat -tulpn
```

Monitoring and Logging

View Resource Usage

```
# Real-time stats
docker stats

# Specific container stats
docker stats <container>

# Memory usage
docker exec <container> free -h

# Disk usage
docker exec <container> df -h
```

Log Management

```
# View logs
docker logs <container>

# Follow logs
docker logs -f <container>

# Limit log output
docker logs --tail 50 <container>

# Logs with timestamps
docker logs -t <container>

# Configure logging driver
docker run --log-driver json-file --log-opt max-size=10m <image>
```

Quick Reference Cards

Dockerfile Instructions Quick Reference

Instruction	Description	Example
FROM	Base image	FROM ubuntu:20.04
MAINTAINER	Image maintainer (deprecated)	MAINTAINER John <john@example.com>

Instruction	Description	Example
LABEL	Image metadata	LABEL version="1.0"
RUN	Execute command	RUN apt-get update
CMD	Default command	CMD ["python", "app.py"]
ENTRYPOINT	Configure container executable	ENTRYPOINT ["python"]
EXPOSE	Inform about ports	EXPOSE 8080
ENV	Environment variable	ENV PATH=/usr/local/bin:\$PATH
ADD	Copy files (with features)	ADD file.tar.gz /app/
COPY	Copy files	COPY . /app
VOLUME	Mount point	VOLUME ["/data"]
WORKDIR	Working directory	WORKDIR /app
USER	User for RUN, CMD, ENTRYPOINT	USER nginx
ARG	Build-time variable	ARG VERSION=latest
SHELL	Default shell	SHELL ["/bin/bash", "-c"]

Cleanup Commands

```
# Remove stopped containers
docker container prune

# Remove unused images
docker image prune

# Remove unused volumes
docker volume prune

# Remove unused networks
docker network prune

# Remove everything unused
docker system prune

# Remove everything (including volumes)
docker system prune --volumes

# Force remove all containers
docker rm -f $(docker ps -aq)

# Remove all images
docker rmi -f $(docker images -q)
```

Docker Run Options Quick Reference

Option	Description	Example
-d	Detached mode	docker run -d nginx
-it	Interactive with TTY	docker run -it ubuntu bash
-p	Port mapping	docker run -p 8080:80 nginx
-v	Volume mounting	docker run -v /host:/container nginx
--mount	Mount (recommended)	docker run --mount type=volume,src=vol,dst=/data nginx
-e	Environment variable	docker run -e VAR=value nginx
--name	Container name	docker run --name myapp nginx
--rm	Auto-remove on exit	docker run --rm nginx
--network	Specify network	docker run --network mynet nginx
--memory	Memory limit	docker run --memory=512m nginx

Option	Description	Example
--cpus	CPU limit	docker run --cpus=1 nginx
--user	Run as user	docker run --user 1000:1000 nginx
--workdir	Working directory	docker run --workdir /app nginx

Volume Mount Types Quick Reference

Type	Syntax	Example	Use Case
Named Volume	-v volume_name:/path	-v mydata:/data	Persistent data
Bind Mount	-v /host/path:/container/path	-v \$(pwd):/app	Development
Anonymous Volume	-v /container/path	-v /data	Temporary data
tmpfs Mount	--tmpfs /path	--tmpfs /tmp	Memory storage

Network Types Quick Reference

Type	Command	Use Case	Features
Bridge	docker network create mynet	Single host	DNS resolution, isolation
Host	docker run --network host	Performance	No isolation, direct host access
None	docker run --network none	Security	No networking
Overlay	Docker Swarm	Multi-host	Cross-host communication

Learning Resources and Next Steps

Recommended Learning Path

- Beginner:** Start with basic Docker commands and simple Dockerfiles
- Intermediate:** Learn Docker Compose and multi-container applications
- Advanced:** Explore Docker Swarm, Kubernetes, and production deployments

Additional Resources

- [Docker Official Documentation](#)
- [Docker Hub](#) - Image registry
- [Play with Docker](#) - Browser-based Docker playground

- [Docker Best Practices](#)

Practice Projects

1. Containerize a simple web application
2. Create a multi-container application with database
3. Set up a CI/CD pipeline with Docker
4. Deploy to production with Docker Swarm or Kubernetes

Conclusion

Docker has revolutionized how we develop, deploy, and manage applications. By understanding containers, images, and the Docker ecosystem, you can:

- **Improve development efficiency** with consistent environments
- **Simplify deployment** with portable containers
- **Scale applications** easily with orchestration tools
- **Enhance security** with isolated processes

Remember: Docker is not a virtualization technology, it's an application delivery technology. Focus on building lightweight, efficient containers that solve real problems.

Start with simple examples, practice regularly, and gradually move to more complex scenarios. The Docker ecosystem is vast, but mastering the fundamentals covered in this guide will give you a solid foundation for your DevOps journey.

This guide provides a comprehensive introduction to Docker containerization. For the latest updates and detailed documentation, always refer to the official Docker documentation.